

## Punto 4.4 - Problema N+1: identificación y causas

### Práctica

**Ejercicio:** Añadir métodos al repositorio del proyecto AceriaData que provoquen el problema N+1 y métodos que lo eviten con Include. Analizar el número de consultas ejecutadas en cada caso con el logging. Comparar el tiempo de ejecución. Diagnosticar el problema N+1 a partir del log.

**Contexto del proyecto:** En el punto 4.3 se estudiaron las variantes de No Tracking, incluyendo AsNoTrackingWithIdentityResolution. En este punto se profundiza en el problema N+1, que es uno de los problemas de rendimiento más comunes en EF Core. Esta identificación se usará en el punto 4.5 para las soluciones al problema N+1.

### Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

### Paso 2: Añadir los métodos del problema N+1 a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerOrdenesConPlanchasN1();
    List<OrdenFabricacion> ObtenerOrdenesConPlanchasSinN1();
    List<OrdenFabricacion> ObtenerOrdenesConDetalleN1();
    List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerOrdenesConPlanchasN1(); → método que provoca el problema N+1 al cargar planchas.

Línea 11: List ObtenerOrdenesConPlanchasSinN1(); → método que evita el problema N+1 al cargar planchas.

Línea 12: List ObtenerOrdenesConDetalleN1(); → método que provoca el problema N+1 al cargar el detalle.

Línea 13: `List ObtenerOrdenesConDetalleSinN1();` → método que evita el problema N+1 al cargar el detalle.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

**Error común:** si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

## | Paso 3: Implementar los métodos del problema N+1 en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasN1()
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    foreach (var orden in ordenes)
    {
        var planchas = _context.PlanchasAcero
            .AsNoTracking()
            .Where(p => p.OrdenId == orden.Id)
            .ToList();

        orden.Planchas = planchas;
    }

    return ordenes;
}

public List<OrdenFabricacion> ObtenerOrdenesConPlanchasSinN1()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

public List<OrdenFabricacion> ObtenerOrdenesConDetalleN1()
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    foreach (var orden in ordenes)
    {
        var detalle = _context.DetallesOrden
            .AsNoTracking()
            .FirstOrDefault(d => d.OrdenId == orden.Id);

        orden.Detalle = detalle;
    }

    return ordenes;
}

public List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: `public List ObtenerOrdenesConPlanchasN1()` → declara el método que provoca el problema N+1.

Línea 3: `var ordenes = _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 5: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 6: `.ToList();` → materializa la consulta.

Línea 8: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 10: `var planchas = _context.PlanchasAcero` → inicia la consulta de planchas.

Línea 11: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 12: `.Where(p => p.OrdenId == orden.Id)` → filtra por orden.

Línea 13: `.ToList();` → materializa la consulta.

Línea 15: `orden.Planchas = planchas;` → asigna las planchas a la orden.

Línea 18: `return ordenes;` → devuelve las órdenes.

Línea 21: `public List ObtenerOrdenesConPlanchasSinN1()` → declara el método que evita el problema N+1.

Línea 23: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 24: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 25: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 26: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 27: `.ToList();` → materializa la consulta.

Línea 30: `public List ObtenerOrdenesConDetalleN1()` → declara el método que provoca el problema N+1 al cargar el detalle.

Línea 32: `var ordenes = _context.OrdenesFabricacion` → inicia la consulta.

Línea 33: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 34: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 35: `.ToList();` → materializa la consulta.

Línea 37: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 39: `var detalle = _context.DetallesOrden` → inicia la consulta del detalle.

Línea 40: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 41: `.FirstOrDefault(d => d.OrdenId == orden.Id);` → filtra por orden.

Línea 43: `orden.Detalle = detalle;` → asigna el detalle a la orden.

Línea 46: `return ordenes;` → devuelve las órdenes.

Línea 49: `public List ObtenerOrdenesConDetalleSinN1()` → declara el método que evita el problema N+1 al cargar el detalle.

Línea 51: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 52: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 53: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 54: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 55: `.ToList();` → materializa la consulta.

**Error común:** si el bucle accede a una propiedad de navegación sin Include, se ejecuta una consulta adicional por cada orden. Se debe usar Include para cargar las entidades relacionadas en una sola consulta.

## | Paso 4: Crear el caso de uso del problema N+1

Crear el archivo src/AceriaData.Application/UseCases/ProblemaN1UseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ProblemaN1UseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ProblemaN1UseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== PROBLEMA N+1 ===");

        DemostrarConN1();
        DemostrarSinN1();
        CompararRendimiento();
    }

    private void DemostrarConN1()
    {
        Console.WriteLine("\n--- Con N+1 (planchas) ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasN1();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
        }
    }

    private void DemostrarSinN1()
    {
        Console.WriteLine("\n--- Sin N+1 (planchas) ---");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerOrdenesConPlanchasSinN1();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count}");
        Console.WriteLine($"Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas");
        }
    }

    private void CompararRendimiento()
    {
        Console.WriteLine("\n--- Comparación de rendimiento ---");

        var cronometroN1 = Stopwatch.StartNew();
        var ordenesN1 = _unidad.Ordenes.ObtenerOrdenesConPlanchasN1();
        cronometroN1.Stop();
```

```

        var cronometroSinN1 = Stopwatch.StartNew();
        var ordenesSinN1 = _unidad Ordenes.ObtenerOrdenesConPlanchasSinN1();
        cronometroSinN1.Stop();

        Console.WriteLine($"Con N+1: {cronometroN1.ElapsedMilliseconds} ms | Órdenes:
{ordenesN1.Count}");
        Console.WriteLine($"Sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms | Órdenes:
{ordenesSinN1.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ProblemaN1UseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo \_unidad; → campo de la unidad de trabajo.

Línea 10: public ProblemaN1UseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: \_unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== PROBLEMA N+1 ==="); → muestra la cabecera.

Línea 19: DemostrarConN1(); → llama al método que provoca el problema.

Línea 20: DemostrarSinN1(); → llama al método que evita el problema.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 24: private void DemostrarConN1() → declara el método.

Línea 26: Console.WriteLine("\n--- Con N+1 (planchas) ---"); → muestra la cabecera.

Línea 28: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 29: var ordenes = \_unidad Ordenes.ObtenerOrdenesConPlanchasN1(); → carga las órdenes con N+1.

Línea 30: cronometro.Stop(); → detiene el cronómetro.

Línea 32: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 33: Console.WriteLine(\$"Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 34: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 36: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas"); → muestra el número de planchas.

Línea 40: private void DemostrarSinN1() → declara el método.

Línea 42: Console.WriteLine("\n--- Sin N+1 (planchas) ---"); → muestra la cabecera.

Línea 44: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 45: var ordenes = \_unidad Ordenes.ObtenerOrdenesConPlanchasSinN1(); → carga las órdenes sin N+1.

Línea 46: cronometro.Stop(); → detiene el cronómetro.

Línea 48: Console.WriteLine(\$"Órdenes: {ordenes.Count}"); → muestra el número de órdenes.

Línea 49: Console.WriteLine(\$"Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 50: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 52: `Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas");` → muestra el número de planchas.

Línea 56: `private void CompararRendimiento()` → declara el método.

Línea 58: `Console.WriteLine("\n--- Comparación de rendimiento ---");` → muestra la cabecera.

Línea 60: `var cronometroN1 = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 61: `var ordenesN1 = _unidad Ordenes.ObtenerOrdenesConPlanchasN1();` → carga las órdenes con N+1.

Línea 62: `cronometroN1.Stop();` → detiene el cronómetro.

Línea 64: `var cronometroSinN1 = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 65: `var ordenesSinN1 = _unidad Ordenes.ObtenerOrdenesConPlanchasSinN1();` → carga las órdenes sin N+1.

Línea 66: `cronometroSinN1.Stop();` → detiene el cronómetro.

Línea 68: `Console.WriteLine($"Con N+1: {cronometroN1.ElapsedMilliseconds} ms | Órdenes: {ordenesN1.Count}");` → muestra el tiempo con N+1.

Línea 69: `Console.WriteLine($"Sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms | Órdenes: {ordenesSinN1.Count}");` → muestra el tiempo sin N+1.

**Error común:** si se ejecutan las dos consultas en el mismo `DbContext`, las entidades de la primera consulta pueden afectar a los resultados de la segunda. Se debe usar un `DbContext` distinto para cada consulta.

## | Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ProblemaN1UseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

**Error común:** si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

## | Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ProblemaN1UseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

**Error común:** si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

## | Paso 7: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```

using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>
    {
        new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0004", Cliente = "Constructora del Oeste", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 4, 5) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0005", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 5, 12) },
        new OrdenFabricacion { NumeroOrden = "OF-2024-0006", Cliente = "Constructora del Sur", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 6, 18) }
    };

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();

    var planchas = new List<PlanchaAcero>
    {
        new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[0].Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[1].Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[2].Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso = 450.0m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[3].Id, Espesor = 9.0, Ancho = 1100, Largo = 2200, Peso = 180.0m, Activa = true },
        new PlanchaAcero { OrdenId = ordenes[4].Id, Espesor = 11.0, Ancho = 1300, Largo = 2700, Peso = 290.0m, Activa = true }
    };

    context.PlanchasAcero.AddRange(planchas);
    context.SaveChanges();
}

```

Línea 1: using (var scope = \_provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var ordenes = new List → crea la lista de órdenes.

Línea 9: new OrdenFabricacion { ... }, → primera orden.

Línea 10: new OrdenFabricacion { ... }, → segunda orden.

Línea 11: new OrdenFabricacion { ... }, → tercera orden.

Línea 12: new OrdenFabricacion { ... }, → cuarta orden.

Línea 13: new OrdenFabricacion { ... }, → quinta orden.

Línea 14: new OrdenFabricacion { ... } → sexta orden.

Línea 15: }; → cierra la lista.

Línea 17: context.OrdenesFabricacion.AddRange(ordenes); → registra las órdenes.

Línea 18: `context.SaveChanges();` → inserta las órdenes.

Línea 20: `var planchas = new List` → crea la lista de planchas.

Línea 22: `new PlanchaAcero { ... }`, → primera plancha.

Línea 23: `new PlanchaAcero { ... }`, → segunda plancha.

Línea 24: `new PlanchaAcero { ... }`, → tercera plancha.

Línea 25: `new PlanchaAcero { ... }`, → cuarta plancha.

Línea 26: `new PlanchaAcero { ... }`, → quinta plancha.

Línea 27: `new PlanchaAcero { ... }` → sexta plancha.

Línea 28: `};` → cierra la lista.

Línea 30: `context.PlanchasAcero.AddRange(planchas);` → registra las planchas.

Línea 31: `context.SaveChanges();` → inserta las planchas.

**Error común:** si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

## Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo con N+1 y sin N+1, y el número de consultas ejecutadas en el log.

## Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== PROBLEMA N+1 ===

--- Con N+1 (planchas) ---
Órdenes: 6
Tiempo: 145 ms
  OF-2024-0001: 2 planchas
  OF-2024-0002: 1 planchas
  OF-2024-0003: 1 planchas
  OF-2024-0004: 1 planchas
  OF-2024-0005: 1 planchas
  OF-2024-0006: 0 planchas

--- Sin N+1 (planchas) ---
Órdenes: 6
Tiempo: 42 ms
  OF-2024-0001: 2 planchas
  OF-2024-0002: 1 planchas
  OF-2024-0003: 1 planchas
  OF-2024-0004: 1 planchas
  OF-2024-0005: 1 planchas
  OF-2024-0006: 0 planchas

--- Comparación de rendimiento ---
Con N+1: 145 ms | Órdenes: 6
Sin N+1: 42 ms | Órdenes: 6
```

La primera sección muestra el tiempo con N+1. La segunda sección muestra el tiempo sin N+1. La tercera sección muestra la comparación. La diferencia es notable: 145 ms con N+1 frente a 42 ms sin N+1.



**Observaciones:** el tiempo con N+1 crece linealmente con el número de órdenes. El tiempo sin N+1 es constante. El log de EF Core muestra el patrón de N+1: una consulta para las órdenes y N consultas para las planchas.

## Paso 10: Diagnosticar un error común

Modificar el método `ObtenerOrdenesConPlanchasSinN1` para usar `AsNoTracking` después del `Include`:

```
public List<OrdenFabricacion> ObtenerOrdenesConPlanchasSinN1()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado: el código funciona correctamente. `AsNoTracking` se puede aplicar antes o después del `Include`. El resultado es el mismo.

Solución: no hay error. `AsNoTracking` se puede aplicar en cualquier punto de la consulta antes de la materialización.

## Errores comunes del ejercicio

| Error                    | Causa                                                      | Solución                                             |
|--------------------------|------------------------------------------------------------|------------------------------------------------------|
| N+1 con carga Lazy       | Se accede a propiedades de navegación en un bucle          | Usar Include                                         |
| N+1 con FirstOrDefault   | Se usa FirstOrDefault en un bucle                          | Usar Include o cargar todas las entidades de una vez |
| N+1 con proyección       | Se proyecta una colección sin ToList                       | Añadir ToList dentro de la proyección                |
| N+1 con segundo nivel    | Se accede a una propiedad de segundo nivel sin ThenInclude | Usar ThenInclude                                     |
| Tiempo alto              | Se ejecutan N+1 consultas                                  | Usar Include                                         |
| Log con muchas consultas | Se ejecuta una consulta por cada entidad                   | Usar Include                                         |

## Reto resuelto: Identificar el problema N+1 en una consulta con detalle

**Reto:** Crear un método en el repositorio que cargue las órdenes con su detalle usando `FirstOrDefault` en un bucle, provocando el problema N+1. Crear otro método que lo evite con `Include`. Comparar el número de consultas ejecutadas y el tiempo.

**Solución paso a paso:**

### Paso 1: Añadir los métodos a la interfaz `IOrdenRepositorio`:

```
List<OrdenFabricacion> ObtenerOrdenesConDetalleN1();
List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1();
```

Línea 1: `List ObtenerOrdenesConDetalleN1();` → declara el método con N+1.

Línea 2: `List ObtenerOrdenesConDetalleSinN1();` → declara el método sin N+1.

## Paso 2: Implementar los métodos en OrdenRepository:

```
public List<OrdenFabricacion> ObtenerOrdenesConDetalleN1()
{
    var ordenes = _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();

    foreach (var orden in ordenes)
    {
        var detalle = _context.DetallesOrden
            .AsNoTracking()
            .FirstOrDefault(d => d.OrdenId == orden.Id);

        orden.Detalle = detalle;
    }

    return ordenes;
}

public List<OrdenFabricacion> ObtenerOrdenesConDetalleSinN1()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Línea 1: public List ObtenerOrdenesConDetalleN1() → declara el método.

Línea 3: var ordenes = \_context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTracking() → aplica AsNoTracking.

Línea 5: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 6: .ToList(); → materializa la consulta.

Línea 8: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 10: var detalle = \_context.DetallesOrden → inicia la consulta del detalle.

Línea 11: .AsNoTracking() → aplica AsNoTracking.

Línea 12: .FirstOrDefault(d => d.OrdenId == orden.Id); → filtra por orden.

Línea 14: orden.Detalle = detalle; → asigna el detalle a la orden.

Línea 17: return ordenes; → devuelve las órdenes.

Línea 20: public List ObtenerOrdenesConDetalleSinN1() → declara el método sin N+1.

Línea 22: return \_context.OrdenesFabricacion → inicia la consulta.

Línea 23: .AsNoTracking() → aplica AsNoTracking.

Línea 24: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 25: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 26: .ToList(); → materializa la consulta.

### Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarDetalleN1()
{
    Console.WriteLine("\n--- Detalle con N+1 ---");
    var cronometroN1 = Stopwatch.StartNew();
    var ordenesN1 = _unidad.Ordenes.ObtenerOrdenesConDetalleN1();
    cronometroN1.Stop();
    Console.WriteLine($"Tiempo con N+1: {cronometroN1.ElapsedMilliseconds} ms");
    foreach (var orden in ordenesN1)
    {
        var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle";
        Console.WriteLine($" {orden.NumeroOrden}: {detalle}");
    }

    Console.WriteLine("\n--- Detalle sin N+1 ---");
    var cronometroSinN1 = Stopwatch.StartNew();
    var ordenesSinN1 = _unidad.Ordenes.ObtenerOrdenesConDetalleSinN1();
    cronometroSinN1.Stop();
    Console.WriteLine($"Tiempo sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms");
    foreach (var orden in ordenesSinN1)
    {
        var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle";
        Console.WriteLine($" {orden.NumeroOrden}: {detalle}");
    }
}
```

Línea 1: private void DemostrarDetalleN1() → declara el método.

Línea 3: Console.WriteLine("\n--- Detalle con N+1 ---"); → muestra la cabecera.

Línea 4: var cronometroN1 = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 5: var ordenesN1 = \_unidad.Ordenes.ObtenerOrdenesConDetalleN1(); → carga las órdenes con N+1.

Línea 6: cronometroN1.Stop(); → detiene el cronómetro.

Línea 7: Console.WriteLine(\$"Tiempo con N+1: {cronometroN1.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 8: foreach (var orden in ordenesN1) → itera sobre las órdenes.

Línea 10: var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle"; → obtiene el detalle.

Línea 11: Console.WriteLine(\$" {orden.NumeroOrden}: {detalle}"); → muestra el detalle.

Línea 14: Console.WriteLine("\n--- Detalle sin N+1 ---"); → muestra la cabecera.

Línea 15: var cronometroSinN1 = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 16: var ordenesSinN1 = \_unidad.Ordenes.ObtenerOrdenesConDetalleSinN1(); → carga las órdenes sin N+1.

Línea 17: cronometroSinN1.Stop(); → detiene el cronómetro.

Línea 18: Console.WriteLine(\$"Tiempo sin N+1: {cronometroSinN1.ElapsedMilliseconds} ms"); → muestra el tiempo.

Línea 19: foreach (var orden in ordenesSinN1) → itera sobre las órdenes.

Línea 21: var detalle = orden.Detalle?.ComposicionQuimica ?? "Sin detalle"; → obtiene el detalle.

Línea 22: Console.WriteLine(\$" {orden.NumeroOrden}: {detalle}"); → muestra el detalle.

### Paso 4: Llamar al método desde Ejecutar:

```
DemostrarDetalleN1();
```

## | Paso 5: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: el tiempo con N+1 es notablemente mayor que el tiempo sin N+1. El log muestra una consulta para las órdenes y N consultas para el detalle.

## | Analogía final

El problema N+1 en una acería es como ir al archivo central a buscar las carpetas de las órdenes y, en lugar de pedir todas las carpetas de planchas de una vez, pedir una carpeta de planchas por cada orden. Si hay diez órdenes, se hacen once viajes al archivo: uno para las órdenes y diez para las planchas. Cada viaje tiene un coste: abrir la puerta, buscar la carpeta, cerrar la puerta y volver. El tiempo total es la suma de todos los viajes. La solución es pedir todas las carpetas de planchas de una vez: un solo viaje con todo lo necesario. Include es como pedir todas las carpetas relacionadas en un solo viaje. ThenInclude es como pedir también las carpetas de las carpetas. El logging es como el registro de todos los viajes que se han hecho al archivo: si se ven muchos viajes idénticos con parámetros distintos, hay un problema N+1. Así funciona el problema N+1 en EF Core: se identifica con el logging, se mide con cronómetros y se resuelve con Include.

## | Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos del problema N+1 a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso ProblemaN1UseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con varias órdenes y planchas.

Ejecutado las demostraciones con N+1 y sin N+1.

Comparado el rendimiento de ambas variantes.

Analizado el log de EF Core y observado el patrón de N+1.

Diagnosticado el error de aplicar AsNoTracking después del Include.

Creado los métodos ObtenerOrdenesConDetalleN1 y ObtenerOrdenesConDetalleSinN1.

---